

REPORT



■ 과목명	딥러닝 응용
■ 교수명	김선만 교수
■ 학과명	소프트웨어융합학부
■ 학 번	202155011
■ 이 름	송영욱
■ 제출일	2021/11/11

Vanilla RNN, LSTM, GRU의 분류 정확도 비교 분석

Vanilla RNN

- 기존의 FeedForward 방식에서 sequence data의 시간적 종속성을 처리하지 못하는 한계가 있어서 RNN이 등장했음.
- RNN 순환관계는 순환의 개념 보다는 시간 순서대로 데이터를 처리하는데 피드백을 다음 순서 데이터에 반영하는 것을 보고 순환이라는 개념을 사용함.
- 상태 벡터를 사용하는데 새로운 은닉 상태는 이전 은닉 상태에 입력 데이터를 합치고 가중치를 곱하여 다음 상태 벡터를 출력합니다. 이러한 방식을 하이퍼볼릭 탄젠트를 이용하여 출력함.
- Vanilla의의미는 가장 기본이 된다는 개념이다. Hidden State에서 정보 업데이트가 이루어진다. 상황, 상태, 맥락 등을 전달하는 단기기억이면서 당자의 문제 해결을 위한 출력함.

Vanilla RNN의 구조

```
self.dimension = dimension
self.hidden_size = hidden_size
self.num_layers = num_layers
self.num_classes = num_classes
self.device = device

# layer 구조
h0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size).to(self.device)
```

Vanilla RNN 모델

```
Number of trainable parameters: 21,514
Input shape: torch.Size([8, 28, 28])
Output shape: torch.Size([8, 10])
```

Vanilla RNN 정확도

Test Accuracy: 96.36%

```
PS C:\Users\User\Desktop\DL_LOG\jo\pr_week_13_torch_mnist_LSTM> & C:/ProgramData/anaconda3/python.exe c:/Users/User/Desktop/DL_LOG/jo/pr_week_13_torch_mnist_LSTM/test_rnn.py
Loading model from: trained_output/rnn_mnist_b32_lr0.001_h64_l1\epoch_25.pth
c:\Users\User\Desktop\DL_LOG\jo\pr_week_13_torch_mnist_LSTM\test_rnn.py:85: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which
module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md
or more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitr
r be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_on
where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
state_dict = torch.load(model_path, map_location=device)
Test Accuracy: 96.36%
[]
```

LSTM

- RNN은 이전의 입력 데이터를 기억해 다음 출력 값을 지정한다. 하지만 short term memory이 이다. 이력데이터가 길어지면 그래디언트 소실 문제(Gradient Vanishing Problem)이 발생하여 이전의 정보를 제대로 기억 못하는 문제가 있음.
- **LSTM**(Long Short-Term Memory = 장단기 기억) 단기기억을 길게 기억하겠다는 의미이다. 하지만 중요한걸 길게 기억하는 관점보다는 쓸모 없는 정보를 얼마나 잘 무시하고 잊어 버리느냐가 핵심이다. 3가지의 Gate 망각 게이트(Forget Gate), 입력 게이트(Input Gate), 출력 게이트(Output Gate)를 통해 기존의 정보를 제어하고 관리함.
- 전체적인 구조는 RNN과 비슷하지만 Cell state 추가하여 기억을 길게 가지고 감.
- **Cell State**는 이전 상태에서 현재 상태까지 유지되는 정보의 흐름을 나타내면, 이를 통해 LSTM은 오래된 정보를 기억하고 새로운 정보를 갱신함.
- **Forget Gate**는 이전 셀 상태의 정보를 지울 것인지 말 것인지를 결정함. 이값을 계산하기 위해 Sigmoid함수가 사용된다. (0~1의 값을 가지게 됨) 값이 0이면 사라지고 1이면 전달함.
- **Input Gate**는 새로운 정보를 어떻게 반영할 것인가에 대한 결정을 내린다. Sigmoid함수와 Tanh을 사용한다. Tanh함수는 기존의 RNN에서 사용되는 출력 계산 방법과 동일하다. Sigmoid 함수는 후보 값을 얼마나 전달 할지 결정 내리는 값이다.이 두값을 곱하고 더하면 다음 단계로 보낼 준비함.
- **Output Gate**는 출력이 얼마나 중요한지 조절하기 위해 은닉 상태와 현재 입력에 대해 시그모이드함수를사용하고 셀상태는 탄젠트함수를 사용하여 출력함.
- 각 게이트는 가중치 행렬이 서로 다르게 학습함.

LSTM의 구조

```
self.dimension = dimension
    self.hidden_size = hidden_size
    self.num_layers = num_layers
    self.num_classes = num_classes
    self.device = device

# layer 구조 (CELL STATE 추가)
h0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size).to(self.device)
c0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size).to(self.device)
```

LSTM 모델

```
Number of trainable parameters: 82,186
Input shape: torch.Size([8, 28, 28])
Output shape: torch.Size([8, 10])
```

LSTM의 정확도

Test Accuracy: 98.31%

```
PS C:\Users\User\Desktop\DL_LOG_jo\pr_week_13_torch_mnist_LSTM> & C:/ProgramData/anaconda3/python.exe c:/Users/User/Desktop/DL_LOG_jo/pr_week_13_torch_mnist_LSTM/test_rnn.py
loading model from: trained_output/lstm_mnist_b32_lr0.001_h64_l1/epoch_13.pth
c:\Users\User\Desktop\DL_LOG_jo\pr_week_13_torch_mnist_LSTM\test_rnn.py:85: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  state_dict = torch.load(model_path, map_location=device)
Test Accuracy: 98.31%
[]
```

GRU

- LSTM의 연산 방식이 너무 복잡해서 Cell state를 다시 없애고, Hidden state가 기억과 출력을 모두 담당하도록 통합함.
- **Reset Gate**, Update Gate 두 게이트를 사용하여 작동함. Reset Gate는 이전 상태를 얼마나 잊어야 하는지를 결정, Update Gate는 이전 상태의 정보를 얼마나 가져와야 할지를 결정 함
- **Reset Gate**(Forget Gate)는 이전 상태의 정보를 유지할지 결정함. Sigmoid함수를 사용해서 정보를 판단함.
- **Update Gate**(Forget Gate+Input Gate)는 이전 상태의 정보와 새로운 정보를 결합하는 것 사이의 균형을 결정함. 입력값과 이전 값이 Sigmoid함수를 통해 이전 상태에서 곱함. Tanh함수를 사용해서 Sigmoid함수 계산값을 곱하여 출력값에 더하여 출력함.

GRU의 구조

```
self.dimension = dimension
    self.hidden_size = hidden_size
    self.num_layers = num_layers
    self.num_classes = num_classes
    self.device = device

# layer 구조
h0 = torch.zeros(self.num_layers, x.size(0), self.hidden_size).to(self.device)
```

GRU모델

Number of trainable parameters: 61,962
Input shape: torch.Size([8, 28, 28])
Output shape: torch.Size([8, 10])

GRU의 정확도

Test Accuracy: 98.65%

```
PS C:\Users\User\Desktop\DL_LOG\jo\pr_week_13_torch_mnist_LSTM> & C:\ProgramData\anaconda3\python.exe c:\Users\User\Desktop\DL_LOG\jo\pr_week_13_torch_mnist_LSTM\test_rnn.py
Loading model from: trained_output/gru_mnist_b32_lr0.001_h64_11\epoch_23.pth
c:\Users\User\Desktop\DL_LOG\jo\pr_week_13_torch_mnist_LSTM\test_rnn.py:92: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle
module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models f
or more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer
be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for any use case
where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
state_dict = torch.load(model_path, map_location=device)
Test Accuracy: 98.65%
```

GRU를 변형하여 만들었습니다.

- 시행착오

hidden_size를 128까지 확장하고 레이어 수를 2~8까지 증가시키면서 모델 깊이에 따른 성능 변화를 분석하였다.

특히,

- hidden_size=128, num_layers=5 조합에서 **99.08%~99.10%**로 높은 성능을 기록하며 유의미한 기준점을 확보했다.

• 대규모 모델 구성 및 성능 정체 구간 분석

모델의 용량을 대폭 확장하여 성능 상향을 시도하였다.

- hidden_size: 256 → 512
- num_layers: 7 → 8

이 실험에서

- 256/7/512/0.001/60/0.1 구성으로 **99.16%**를 달성하며 해당 구간의 최고 성능을 기록하였다. 그러나 모델 폭을 512까지 확장한 실험에서는 성능이 오히려 98%대까지 하락하는 현상이 관찰되었다. 이는 다음을 시사한다.

1. 과도한 모델 복잡도 → 과적합 발생 가능성 증가
2. 최적의 성능이 이미 이전 단계에서 결정되었으며 단순 모델 크기 확장은 효과적이지 않음
3. 학습 안정성이 떨어지며 검증 손실 변동성이 증가

즉, 모델의 크기를 무작정 늘리는 접근은 한계가 분명함을 확인하였다.

• 드롭아웃 세부 조정 및 옵티마이저 변경 실험

일부 실험에서는 특정 레이어에만 드롭아웃을 적용하거나 제거하는 등 비표준적인 정규화 조절을 시도하였다.

예:

- 3번째 layer dropout 미적용
- 5번째 layer dropout 0
- 2번째 layer dropout 조정

이는 PyTorch의 기본 nn.GRU 구조에 제공되는 레이어 간 드롭아웃 외에, 세밀한 제어를 통해 과적합 억제 효과를 극대화하기 위한 실험이었다.

옵티마이저 실험

- 기본: Adam
- 대체 탐색: AdamW

AdamW는 가중치 감쇠 구조 상 안정성을 제공할 것으로 기대되었으나, 최종적으로 Adam 대비 유의미한 성능 향상은 제한적이었다.

- **최종 최적화 전략 및 최고 성능 달성**

여러 모델 확장 실험을 거친 끝에, 모델의 폭을 과하게 키우는 대신 깊이(Depth) 를 증가시키는 전략이 가장 효과적이었다.

최종 선택된 최적 조합:

- hidden_size = 128
- num_layers = 10
- batch_size = 128
- learning rate = 0.001
- epochs = 60
- dropout = 0.1

이 구성에서 ****99.20%****의 최고 정확도를 달성하였다.

성능 향상의 결정적 요인: 학습률 스케줄러(LR Scheduler)

레이어 수 증가로 인해 모델이 매우 깊어지면서 다음 문제가 발생했다.

1. 과적합 경향 증가
2. 검증 손실이 일정 수준에서 정체되는 현상
3. 연산량 증가로 학습 시간이 급격히 증가

이를 해결하기 위해 검증 손실이 감소하지 않을 때 학습률을 동적으로 증가시키는 스케줄러를 적용하였다. 이 전략은 기울기 소실로 인해 학습이 정체되는 구간을 탈출하게 하여 최종 성능 향상에 크게 기여했다.

- 모델 구조 설계안 제시

구성 요소 (Component)	세부 내용 (Detail)	설명 (Description)
모델 타입	GRU (Gated Recurrent Unit)	PyTorch nn.GRU 모듈 사용
순환층 깊이 (num_layers)	10	모델의 깊이(Depth)
은닉층 크기 (hidden_size)	128	은닉 상태 벡터의 차원
입력 형태 (batch_first)	True	입력 텐서 형태: (Batch, Sequence, Feature)
드롭아웃 (dropout)	0.1	다중 레이어 간 드롭아웃 적용
최종 출력층	<code>nn.Linear(128, num_classes)</code>	최종 시점의 은닉 상태를 입력으로 받는 완전 연결층
출력 값	<code>num_classes</code> (10으로 추정)	분류 문제의 클래스 개수

- 학습 하이퍼파라미터 제시

항목 (Parameter)	값 (Value)	비고 (Note)
Hidden Size	128	최적의 은닉층 크기
Number of Layers	10	최적의 모델 깊이
Batch Size	128	배치 크기
Learning Rate (LR)	0.001	초기 학습률
Epochs	60	총 학습 횟수
Dropout	0.1	nn.GRU 레이어 간 드롭아웃 비율
손실 함수 (Criterion)	<code>nn.CrossEntropyLoss()</code>	다중 클래스 분류를 위한 손실 함수
옵티마이저 (Optimizer)	<code>torch.optim.Adam</code>	안정적인 수렴을 위한 옵티마이저
스케줄러 (Scheduler)	<code>ReduceLROnPlateau</code>	학습률 감소 스케줄러 ('min', factor=0.5, patience=3)

- 최종 결과 제시

Test Accuracy: 99.20%

```
test_rnn.py X gruconv.py gruconv.py train_gru.py
test_rnn.py > ...
58 def main(args):
59     return
60     # 테스트 수행
61     test_acc, y_pred = test(model, test_loader, device=device)
62     print("Test Accuracy: {:.2f}%".format(test_acc))
63     # 샘플 시각화
64     show_sample(x_test_raw, y_test, y_pred)
65
66 # 실행 | 위에서 실행 | 디버그 실행
67
68 if __name__ == "__main__":
69     parser = argparse.ArgumentParser()
70
71     # Model hyperparameters
72     parser.add_argument("--data-dir", type=str, default="data")
73     parser.add_argument("--hidden-size", type=int, default=128)
74     parser.add_argument("--num-layers", type=int, default=2)
75
76     # Test parameters
77     parser.add_argument("--test-dir", type=str, default="test")
78     parser.add_argument("--batch-size", type=int, default=100)
79     parser.add_argument("--device", type=str, default="cpu")
80
81     args = parser.parse_args()
82
83     # Main logic
84     ...
85
86 # Conda environment info
87 + CategoryInfo          : ObjectNot
88 + FullyQualifiedErrorId : CommandNot
89
90 PS C:\Users\User\Desktop\DL_L06_jo\pr_week_13_torch_mnist_LSTM> & C:\ProgramData\anaconda3\python.exe c:\Users\User\Desktop\DL_L06_jo\pr_week_13_torch_mnist_LSTM/test_rnn.py
c:\Users\User\Desktop\DL_L06_jo\pr_week_13_torch_mnist_LSTM\test_rnn.py:120: SyntaxWarning: invalid escape sequence '\g'
  parser.add_argument("--target model dir", type=str, default="trained output\gru_mnist_b128_lr0.001_h128_l2")
Loading model from: trained_output\gru_mnist_b128_lr0.001_h128_l2\epoch_18.pth
c:\Users\User\Desktop\DL_L06_jo\pr_week_13_torch_mnist_LSTM\test_rnn.py:92: FutureWarning: You are using 'torch.load' with 'weights_only=False' (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for 'weights_only' will be flipped to 'True'. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowedlist by the user via 'torch.serialization.add_safe_globals'. We recommend you start setting 'weights_only=True' for any use case where you don't have full control of the loaded file. Please open a
n issue on GitHub for any issues related to this experimental feature.
  state_dict = torch.load(model_path, map_location=device)
Test Accuracy: 99.20%
[]
```

